Advanced Text Processing and Introduction to Shells and Scripting

Lecture overview

- Text processing in Linux – recap ←
- Searching for text patterns (`grep`)
- Text substitution – search and replace (`sed`)
- Shells and scripting
- Variables in `bash` and capturing command output

Advanced text parsing

Until now we saw simple file manipulations:

- Subset lines (`head`, `tail`)
- Concatenate vertically (`cat`)
- Subset columns (`cut`)
- Concatenate horizontally (`paste`)
- Simple character replacement (`tr`)

The true power of text manipulation in Linux/UNIX comes from commands that use **regular expression** for search and replace operations

Regular expressions

- Regular expressions (regexp) are useful constructs describing families of strings (patterns)
- Several UNIX/Linux text utilities use regexp
- We will learn two regexp-powered commands:
  - `grep` – searching for a specific pattern
  - `sed` – performing a search and replace
- We will discuss several theoretical concepts related to regexp:
  - Connection to finite state machines
  - Matching strategies

Lecture overview

- Text processing in Linux – recap
- Searching for text patterns (`grep`) ←
- Text substitution – search and replace (`sed`)
- Shells and scripting
- Variables in `bash` and capturing command output

`grep` – searching for matching patterns

- The `grep` command searches files for patterns defined by regular expressions, and **prints matching lines**

```
grep [options] regexp [files]
```

- `grep` = **g**lobally search a **r**egular **e**xpression and **p**rint

grep – searching for specific string

The simplest regular expression is just a sequence of characters, designating an identity match to that string

- Example:

```
grep Peter phone_dir.txt
```

```
BENNETT, Peter 7456
HARMER, Peter 7484
MAKORTOFF, Peter 7328
```

Matching a character class

- `.` – matches **any single character**
- `[]` – match any single character within the brackets
- `-` – used to indicate a range, such as `[a-z]` or `[3-6]`
- `^` – at the beginning of the range indicates any character **not** in the brackets
  Example: `[^0-9]` indicates any **non-digit** character

Matching a character class

Examples:

RegExp	Matches	Doesn't Match
`b.g`	`bag` `debugging`	`brag` `bg`
`[Bb]ill`	`Bill` `got billed`	`kill` `ill`
`t[aeiou].k`	`talk track` `stink`	`track` `take`
`number [^0-5]`	`number xxx` `number 8:`	`number 59`
`[B^]`	`Bill` `hello ^`	
`.`	`Any and all characters`	`An empty line`

Character matches – special cases

We can use predefined character classes (same as for `tr`):

- For example, `[:alpha:]` is typically preferable to `[A-Za-z]`
- Note: the brackets are part of the symbolic names, and must be included (in addition to the brackets that enclose a range used in a grep command)
- Example: search lines that contain underscores or letters

```
grep [_[:alpha:]] file
```

We can match characters that already have a special meaning (e.g., `\ . [ ] *`):

- Precede the character with a '\'
- Or use square brackets – any character inside is taken literally
- Example: search lines that contains a seq of two dots, followed by any character and a back slash:

```
grep "\.\..[\]" file
```

Matching repetitions

An asterisk (*) represents *zero* or more matches of the regular expression it follows

RegExp	Matches	Doesn't Match

`ab*c`	`ac`
	`abc`
	`aaabbbccc`
		`abdc`

`t.*ing`	`eating`
	`thing`
	`string`
	`thinking`
		`king`

Match start/end of line

Position of match in line:

- RegExp beginning with `^`: match must start at beginning of line
- RegExp ending with `$`: match must end at end of line

Examples:

RegExp	Matches	Doesn't Match
`^T`	`This line`	`my Tag`
`\[[\]\]$`	`Score: [\]`	`Score: [\] 88`

`^num.*[0-7]$`	`num5`
	`number1`
	`number 1`
		`my num1`
		`num 6a`

`^t[^0-9]*s$`	`this` `ts`	`stacks` `take 4 books`

Command line options for `grep`

Select options:

- `-v` – print all lines that *don't* match
- `-c` – print only a count of matched lines
- `-n` – print line numbers
- `-h` – don't print file names (used for when working with multiple files)
- `-l` – print file name but not matching line
- `--color` – marks the match within the line in color
- `-E` – for extended regular expressions (to be detailed shortly)
- `-F` – for fast matching of regular expressions (to be detailed shortly)

grep – command line option examples

```
cat bugs.txt
```

```
big boy
bad bug
bag
bigger bag
better
boogie nights
```

```
grep 'b.*g.' bugs.txt
```

```
big boy
bigger bag
boogie nights
```

```
grep -v 'b.*g.' bugs.txt
```

```
bad bug
bag
better
```

```
grep -n 'b.g.' bugs.txt
```

```
1:big boy
4:bigger bag
```

```
grep -vn 'b.g.' bugs.txt | head -n1
```

```
2:bad bug
```

Extended regular expressions

`grep -E` (or the deprecated `egrep`) provides extended regular expressions

- It allows for more repetition types (not just '`*`' to signify any number of appearances), e.g.:
  - `+` – at **least** one (= one or more) appearance
  - `?` – at **most** one (= zero or one) appearances
  - `{n}` – exactly `n` appearances
  - `{n,}` – at **least** `n` appearances
  - `{,n}` – at **most** `n` appearances
  - `{n,m}` – between `n` and `m` appearances
- Also:
- `()` – grouping (to apply '`*`', '`+`', '`?`' to a string with more than 1 char)
- `a|b` – matches either regular expression `a` OR regular expression `b`

- `+` – at **least** one (= one or more) appearance
- `?` – at **most** one (= zero or one) appearances
- `{n}` – exactly `n` appearances
- `{n,}` – at **least** `n` appearances
- `{,n}` – at **most** `n` appearances
- `{n,m}` – between `n` and `m` appearances

Extended regular expressions

RegExp	Matches	Doesn't Match
`num6+`	`num666` `num654`	`num566` `num`
`num(60)?5`	`num605` `num555`	`num65` `num60605`
`16{3,5}0`	`166605` `1666660`	`162660` `16666660`
`Barret|Bennet`	`Barret` `Bennet`	`Barnet`
`B(arr|enn)et`	`Barret` `Bennet`	`Barr` `Barrennet`

Fast matching of single string

`grep -F` (or the deprecated `fgrep`) can be used for speed optimized fixed searches of strings

```
grep -F "Peter" phone_dir.txt
```

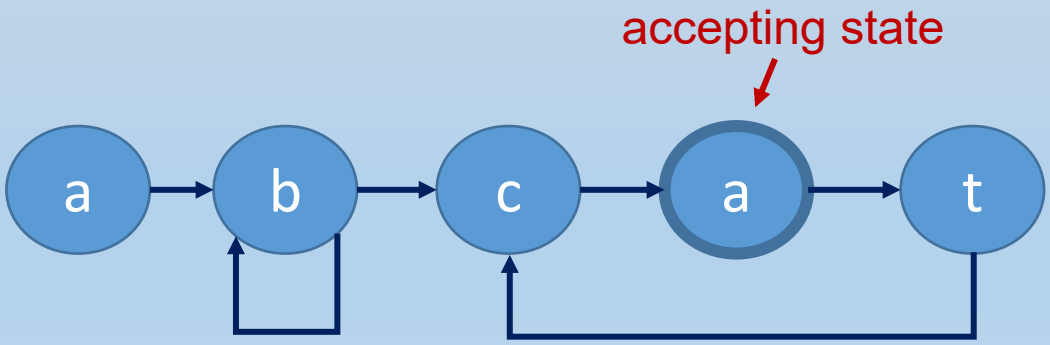
- Quicker (for long files) because it uses a simpler matching algorithm

Regular expressions and finite automata

Which sets of words can be represented by a regular expression?

- any finite set of words
- Infinite sets that can be produced by a finite state machine (automaton)

Regexp: `ab+ca(tca)*`



Regular expressions and finite automata

Are there sets of words that cannot be represented by a regular expression?

- Infinite sets that require unbounded memory
- Example: sequence of a's followed by a sequence of b's of the exact same length

  `ab , aabb , aaabbb , . . .`

Conclusion: regular expressions are very expressive, but there are some fairly simple patterns that they cannot express

Regexp vs. filename expansion

Regular expressions are similar to filename expansion, but they work differently

- Filename expansion is done by the shell
  Example: try typing `echo *` in command line
  - Notice different use for `*`
- Regular expressions are passed as arguments to specific commands or utilities (e.g., `grep` or `sed`)

Regexp vs. filename expansion

Regular expressions are similar to filename expansion, but they work differently

Example: sort C code files (files ending with .c or .h) by number of lines

Solution 1: (filename expansion)

```
wc -l *.[ch] | sort -k1,1nr
```

Solution 2: (regexp)

```
wc -l * | grep '\.[ch]$' | sort -k1,1nr
```

Lecture overview

- Text processing in Linux – recap
- Searching for text patterns (`grep`)
- Text substitution – search and replace (`sed`) ←
- Shells and scripting
- Variables in `bash` and capturing command output

sed – stream editor

- **sed** is a **s**tream **ed**itor for text, which can perform transformations on an input stream, based on regular expressions and simple instructions
- **sed** has several command forms. We will focus on the substitute command ('s') applied in the following format:

```
sed 's/pattern/replacement/[g]' [file]
```

sed – stream editor

```
sed [options] 's/pattern/replacement/' [file]
```

- If a line has a match for **pattern**, the first of the matching strings is replaced by **replacement**
  (more details later for cases with multiple matches)
- If a line does not have a match for **pattern**, it is printed as is
- Modified version of **file** is printed by default to **stdout**
- File can be directly edited by using the **-i** option
  Useful, but risky!
- We will use the **-r** option for extended regular expressions

Reichman University

ADAMS, Andrew 7583
BARRETT, Bruce 6466
BAYES, Ryan 6585
BECK, Bill 6346
BENNETT, Peter 7456
DAVIS, John 3456
ESSA, Konrad 3549
GRAHAM, Linda 6141
HARMER, Peter 7484
LINCOLN, Abe 9749
MAKORTOFF, Peter 7328
MEASDAY, David 6494
NAKAMURA, Satoshi 6453
REEVE, Shirley 7391
ROSNER, David 6830

sed – examples

- Change `','` delimiter to `';'`:

```
sed -r 's/,/;/' phone_dir.txt | head -n1
```

```
ADAMS; Andrew 7583
```

- Remove `','` delimiter:

```
sed -r 's/,//' phone_dir.txt | head -n1
```

```
ADAMS Andrew 7583
```

- Mask phone number with ##:

```
sed -r 's/[[:digit:]]+$/##/' phone_dir.txt | head -n1
```

```
ADAMS, Andrew ##
```

INCORRECT command

```
sed -r 's/[[:digit:]]/##/' phone_dir.txt | head -n1
```

Faulty output

```
ADAMS, Andrew ##583
```

Matching and reusing portions of a pattern

- The special character '**&**' can be used inside **replacement** to refer to the matched text
- Example: enclosing first name between "< " and " >":

```
sed -r 's/ [[:upper:]][[:lower:]]* / <&> /' phone_dir.txt \
       | head -n2
```

```
ADAMS, < Andrew > 7583
BARRETT, < Bruce > 6466
```

INCORRECT command

```
sed -r 's/[[:upper:]][[:lower:]]*/ <&> /' phone_dir.txt \
       | head -n2
```

Faulty output

```
<A> DAMS, Andrew 7583
<B> ARRETT, Bruce 6466
```

Matching and reusing portions of a pattern

- It is also possible to use portions of the matching pattern by enclosing it in parentheses '`(` … `)`' and referring to portion by index in `replacement`, ('`\1`', '`\2`', etc.)
- Example: replacing first name with initial:

```
sed -r 's/ ([[:upper:]])[[:lower:]]* / \1. /' phone_dir.txt | head -n2
```

ADAMS, A. 7583
BARRETT, B. 6466

- Example: switch between first & last names, delete comma:

```
sed -r 's/(.*), (.*) /\2 \1 /' phone_dir.txt | head -n2
```

Andrew ADAMS 7583
Bruce BARRETT 6466

- Example: duplicate numbers so that phone numbers are 8 digits:

```
sed -r 's/(.*) ([[:digit:]]*)/\1 \2\2/' phone_dir.txt | head -n2
```

Andrew ADAMS 75837583
Bruce BARRETT 64666466

Combining `grep` and `sed`

- Lines with no matches are printed as is (not filtered out)
- Often `sed` is combined with `grep` to filter and modify

Example: print all last names and phone numbers of people named Peter

```
grep ', Peter ' phone_dir.txt | \
       sed -r 's/^(.*), Peter (.*)$/\1: \2/'
```

```
BENNETT: 7456
HARMER: 7484
MAKORTOFF: 7328
```

INCORRECT command

```
sed -r 's/^(.*), Peter (.*)$/\1: \2/' phone_dir.txt | \
        grep ', Peter '
```

Faulty output

No output. Why?

Matching policy for regular expressions

What happens if a line contains several overlapping matches of a pattern?

Example:

- regexp: `i.*s`
- line: `Mississippi`
- What is matched?

```
Mississippi
Mississippi
Mississippi
Mississippi
Mississippi
Mississippi
Mississippi
```

- When searching matches with `grep` we only care if a line contains a match or not, so it doesn't matter
- When replacing matches with `sed` it matters

Matching policy for `sed`

- Answer: regexp is matched using a **greedy** policy
  - among all possible matches, take the one that starts first
  - among all matches that start first, take the longest match

Example 1:

- regexp: `i.*s`
- line: `Mississippi`
- What is matched?

```
Mississippi
Mississippi
Mississippi
Mississippi
Mississippi  ←
Mississippi
Mississippi
```

```
echo "Mississippi" | sed -r 's/i.*s/X/'
```

Reichman University

ADAMS, Andrew 7583
BARRETT, Bruce 6466
BAYES, Ryan 6585
BECK, Bill 6346
BENNETT, Peter 7456
DAVIS, John 3456
ESSA, Konrad 3549
GRAHAM, Linda 6141
HARMER, Peter 7484
LINCOLN, Abe 9749
MAKORTOFF, Peter 7328
MEASDAY, David 6494
NAKAMURA, Satoshi 6453
REEVE, Shirley 7391
ROSNER, David 6830

Matching policy for `sed`

- Answer: regexp is matched using a **greedy** policy
  - among all possible matches, take the one that starts first
  - among all matches that start first, take the longest match

Example 2:

```
sed -r 's/[A-Z]*/Doe/' phone_dir.txt | \
   sed -r 's/[6-9]/0/'
```

```
Doe, Andrew 0583
Doe, Bruce 0466
...
```

Reichman University

ADAMS, Andrew 7583
BARRETT, Bruce 6466
BAYES, Ryan 6585
BECK, Bill 6346
BENNETT, Peter 7456
DAVIS, John 3456
ESSA, Konrad 3549
GRAHAM, Linda 6141
HARMER, Peter 7484
LINCOLN, Abe 9749
MAKORTOFF, Peter 7328
MEASDAY, David 6494
NAKAMURA, Satoshi 6453
REEVE, Shirley 7391
ROSNER, David 6830

Matching policy for sed

- Answer: regexp is matched using a **greedy** policy
  - among all possible matches, take the one that starts first
  - among all matches that start first, take the longest match

Example 3:

```
sed -r 's/[^[:space:]]*/-| MATCH="&" |-/' \
   phone_dir.txt
```

```
-| MATCH="ADAMS," |- Andrew 7583
-| MATCH="BARRETT," |- Bruce 6466
 . . .
```

Reichman University

ADAMS, Andrew 7583
BARRETT, Bruce 6466
BAYES, Ryan 6585
BECK, Bill 6346
BENNETT, Peter 7456
DAVIS, John 3456
ESSA, Konrad 3549
GRAHAM, Linda 6141
HARMER, Peter 7484
LINCOLN, Abe 9749
MAKORTOFF, Peter 7328
MEASDAY, David 6494
NAKAMURA, Satoshi 6453
REEVE, Shirley 7391
ROSNER, David 6830

Matching policy for `sed`

Multiple matches can be replaced by adding the `'g'` suffix

```
sed -r 's/pattern/replacement/g' [file]
```

- The greedy strategy is applied repeatedly

Example:

```
sed -r 's/[6-9]/0/g' phone_dir.txt
```

```
ADAMS, Andrew 0503
BARRETT, Bruce 0400
...
```

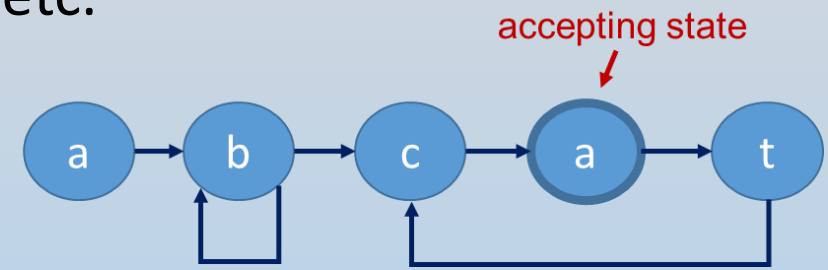
```
sed -r 's/[^[:space:]]*/-| MATCH="&" |-/g'
   phone_dir.txt
```

```
-| MATCH="ADAMS," |- -| MATCH="Andrew" |- -| MATCH="7583" |-
-| MATCH="BARRETT," |- -| MATCH="Bruce" |- -| MATCH="6466" |-
...
```

Regular expressions summary

- Regular expressions are useful for describing patterns to search (and replace) in files
- Regular expressions describe “structured” sets of strings with repetitions, char choices, etc.

Regexp: `ab+ca(tca)*`



- Command `grep` prints lines of file that contain matches with regexp
- Command `sed` can be used to replace matches with other text
  - Can also be used to change order of parts of match
  - Matching policy matters (`sed` uses a greedy policy)

Lecture overview

- Text processing in Linux – recap
- Searching for text patterns (`grep`)
- Text substitution – search and replace (`sed`)
- Shells and scripting ←
- Variables in `bash` and capturing command output

UNIX Shells

- A *shell* is a command interpreter, which acts as an interface between the user and the operating system
- Capabilities:
  - pass commands to the OS (e.g., `cp`, `sed`)
  - define and use variables
  - control flow commands for conditional execution and loops
- Linux has several different shell languages, each with different syntax for variables and control flow commands

➔ Your command line has its own language !!

Common shell languages

Shell	Location	Command
Bourne shell	/bin/sh	sh
C shell	/bin/csh	csh
Bourne Again shell	/bin/bash	bash
Korn shell	/usr/bin/ksh	ksh
Turbo C shell	/bin/tcsh	tcsh

- In this course, we see `bash`, which is the most commonly used shell for Linux

Shell scripts

Scripts are useful for **saving command sequences** for future use (and documentation)

- Any command entered in the command line can also be used in a script
- To run a script, use `source` command:

```
source script_file
```

- We will discuss other ways of executing scripts next week

Script example – 1

- Any command entered in the command line can also be used in a script
- For example, the script `most-freq-word1.sh` performs the operations required for the two steps of the class assignment:

```
mkdir -p ~/classes/class03
cp /share/classes/class03/* ~/classes/class03
cd ~/classes/class03
cat grep-examples.txt |\
    tr [:space:] "\n" |\
    tr -s "\n" |\
    tr "a-z" "A-Z" |\
    sort | uniq -c |\
    sort -k1,1nr | head -n1 |\
    tr -s " " | cut -d" " -f3
```

- To run this script:

```
==> source most-freq-word1.sh
```

The shell interpreter

Shell languages use an interpreter (no compiler)

- Program is “compiled” as it is run
- No binary code is generated
- Syntax error are found on the fly and messages are typically difficult to understand
- Benefit: you can interface directly with the interpreter through the command line (interactive mode)

Other scripting languages also use an interpreter (e.g., `perl`, `python`), but they are not shells (cannot directly run Linux commands)

Lecture overview

- Text processing in Linux – recap
- Searching for text patterns (`grep`)
- Text substitution – search and replace (`sed`)
- Shells and scripting
- Variables in `bash` and capturing command output ←

Shell variables

- Scripts are useful for holding sequences of commands for future use and documentation
- Variables are very useful in this context
- Shell variables are typically untyped (`bash` specifically)
- Default treatment of variables is as strings
- They can also be treated as numbers, and can be used for arithmetic operations (next week)

Defining variables

- To define a variable, we simply assign some value to it
- Assigning values to variables is done as follows:

```
variable=value
```

!! no spaces here !!

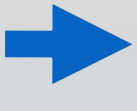
- A variable is referenced using `$`

```
a=hello
b=world
echo $a $b a b
```

```
hello world a b
```

Variables - examples

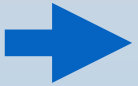
```
name="Bill Clinton"
echo $name
```



Bill Clinton

- Strings with spaces can be assigned by wrapping with " "

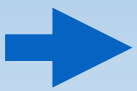
```
echo $SHELL
```



/bin/bash

- Upper case letter variables are typically reserved for environment variables
- The `printenv` command lists all variables defined in current shell

```
echo $some_variable
```



- Non-existing variables are assigned an empty string

```
mkdir -p ~/classes/class03
cp /share/classes/class03/* ~/classes/class03
cd ~/classes/class03
cat grep-examples.txt |\
    tr [:space:] "\n" |\
    tr -s "\n" |\
    tr "a-z" "A-Z" |\
    sort | uniq -c |\
    sort -k1,1nr | head -n1 |\
    tr -s " " | cut -d" " -f3
```

Script example – 1

Script example – 2

- The script `most-freq-word2.sh`:

```
sourceDir=/share/classes/class03
targetDir=~/classes/class03
file=grep-examples.txt

mkdir -p $targetDir
cp $sourceDir/* $targetDir
cd $targetDir
cat $file |\
    tr [:space:] "\n" | tr -s "\n" |\
    tr "a-z" "A-Z" | sort | uniq -c |\
    sort -k1,1nr | head -n1 |\
    tr -s " " | cut -d" " -f3
```

- Variables store the source and target directories and the file name

- This enables easy modification of these components in the script (next week we'll see how to pass script arguments)

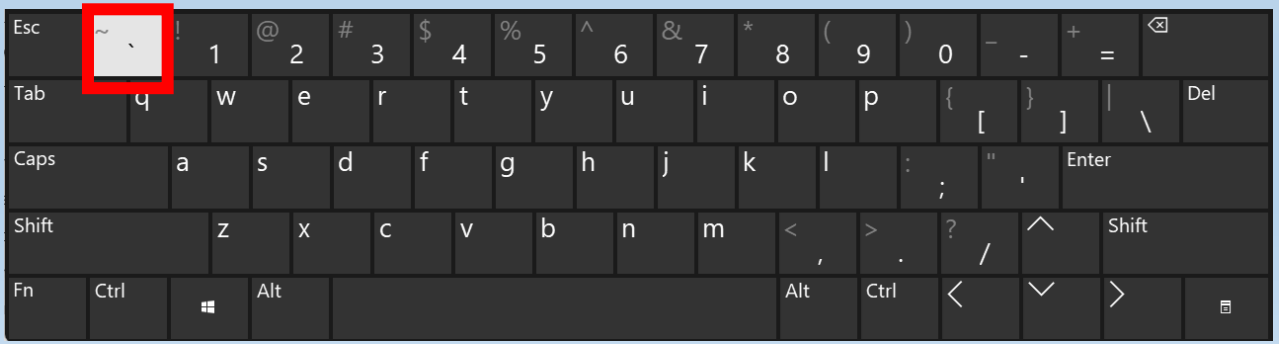
Capture command output

The text output of a command can be captured from `stdout` by enclosing the full command with backquotes (`)

- Can be used to direct text into a variable:

```
name=Satoshi
number=`grep $name phone_dir.txt | cut -d" " -f3`
echo $name\'s phone number is: $number
```

```
Satoshi's phone number is: 6453
```



Capture command output

The text output of a command can be captured from `stdout` by enclosing the full command with backquotes (`)

- Can be used to direct text into a variable:

```
name=Satoshi
number=`grep $name phone_dir.txt | cut -d" " -f3`
echo $name\'s phone number is: $number
```

```
Satoshi's phone number is: 6453
```

- Can also be directly used in another command:

```
name=Satoshi
echo $name\'s phone number is: \
 `grep $name phones.txt | cut -d" " -f3`
```

Script example – 3

- The script `most-freq-word3.sh`:

```
sourceDir=/share/classes/class03
targetDir=~/classes/class03
file=grep-examples.txt

mkdir -p $targetDir
cp $sourceDir/* $targetDir
cd $targetDir
freq_word=`cat $file |\
    tr [:space:] "\n" | tr -s "\n" | tr "a-z" "A-Z" |\
    sort | uniq -c |
    sort -k1,1nr | head -n1 |\
    tr -s " " | cut -d" " -f3`

echo "The most frequent word in $file is $freq_word"
```

- We store the most frequent word in a variable and print it later

Control flow in `bash`

- As in other programming languages, shell languages have control flow commands:
  - Branching statements (if, else)
  - Loops (for, while)
- Review syntax for these statements in the guide on Piazza

Script example – 4

- The script `most-freq-word4.sh`:

```
sourceDir=/share/classes/class03
targetDir=~/classes/class03
file=grep-examples.txt

if [ ! -d $sourceDir ]; then    # check if source dir exists
   echo "Error: source directory $sourceDir does not exist."
else
   if [ ! -d $targetDir ]; then # check if target dir exists
      echo "Creating target dir $targetDir."
      mkdir -p $targetDir
   else
      echo "Target dir $targetDir already exists."
   fi
   echo "Copying  contents of $sourceDir to $targetDir."
   cp $sourceDir/* $targetDir
   cd $targetDir
   . . .
```

- We use if statements to check existence of source / target directory

Script example – 4

- The script `most-freq-word4.sh`:

```
 . . .

   files=`ls *.txt`    # store list of *.txt files in var $files
   for file in $files; do    # iterate over files
      freq_word=`cat $file |\
          tr [:space:] "\n" |\
          tr -s "\n" |\
          tr "a-z" "A-Z" |\
          sort | uniq -c |\
          sort -k1,1nr | head -n1 |\
          tr -s " " | cut -d" " -f3`
      echo "  -> The most frequent word in $file is $freq_word"
   done
fi

echo "Done."
```

- We use a for loop to iterate over all files

At home…

- Homework exercise #2 due Sunday, April 12 @ 21:00
- Homework exercise #3 will be published on Thursday
- Read guide on `bash` control flow statements. We will need this next lecture, and we will not review this closely in class.